Lab 1: Introduction to Natural Language Processing

Session 3: Static word-embeddings for text representation.

Goal

The main goal of this session is to explore the techniques utilized to generate dense vector representations of words, commonly known as word embeddings. It aims to familiarize students with techniques such as Word2Vec, GloVe, and FastText, essential to creating static word embeddings. Students can build document representations based on these techniques, thus enhancing their

understanding and proficiency in text representation for addressing natural language processing tasks.

Bibliography

1. Mikolov, Tomas, Kai Chen, Greg Corrado, Jeffrey Dean. "Efficient estimation of word representations in vector space." 1st International Conference on Learning Representations, ICLR 2013. Workshop Track Proceedings (2013). https://arxiv.org/abs/1301.3781. https://github.com/tmikolov/word2vec
2. Pennington, Jeffrey, Richard Socher, and Christopher D. Manning. "Glove: Global vectors for word representation." *Proceedings of the 2014 conference on empirical methods in natural language processing (EMNLP)*. 2014. https://aclanthology.org/D14-1162.pdf https://nlp.stanford.edu/projects/glove/
3. Bojanowski, Piotr, et al. "Enriching word vectors with subword information." *Transactions of the Association for Computational Linguistics* 5 (2017): 135-146. https://aclanthology.org/Q17-1010.pdf https://fasttext.cc/
4. Gensim https://github.com/piskvorky/gensim#documentation

WORD EMBEDDINGS

Word embeddings are numerical representations of words in a continuous vector space, where words with similar meanings are positioned closer to each other. Related words like "*women*" and "*men*" are assigned to very close n-dimensional vectors, while different words like "*men*" and "*overweight*" have distant vectors. The meaning of a word can be reflected in its vector (embedding), and a model can use this information to learn the relationship between words. Word embedding methods are based on *the*

principle of distributional semantics, which assumes that words occurring in similar contexts have similar meanings. These embeddings capture semantic relationships between words, allowing machines to interpret and process language more effectively. It can be distinguished between two main types of word embeddings: contextual and static.

Static Word Embeddings (SWE)

Static embeddings assign a fixed vector representation to each word in the vocabulary, regardless of its context within a sentence or document. Techniques like Word2Vec, GloVe, and FastText are grouped in this category. **While static embeddings capture general semantic meanings of words, they fail to capture nuances that arise from varying contexts**.

ALGORITHMS FOR BUILDING STATIC WORD EMBEDDING

Word2Vec is a popular algorithm used to generate word embeddings, which are dense vector representations of words in a continuous vector space. It consists of two main models: Continuous Bag of Words (CBOW) and Skip-gram.

Continuous Bag of Words (CBOW):
CBOW predicts the target word based on the context words surrounding it. It takes a context window of surrounding words and tries to predict the target word in the middle. The model learns to predict the target word by minimizing the difference between the predicted probability distribution and the actual distribution of words.

Skip-gram:

Skip-gram, however, predicts the context words given a target word. It takes a target word and tries to predict the context words that will appear around it. The model learns to predict the context words by maximizing the probability of observing them given the target word.

Both CBOW and Skip-gram are trained using a large corpus of text data and learn to generate word embedding vectors that capture semantic relationships between words based on their co-occurrence patterns in the text.

FastText is an extension of the Word2Vec algorithm that Facebook's AI Research (FAIR) team introduced. It enhances Word2Vec by incorporating subword information to handle out-of-vocabulary words more effectively and to capture morphological details. In FastText, words are represented as bags of character n-grams, allowing the model to handle unseen words by leveraging character-level information. Subword embeddings are combined to form word embeddings, enabling FastText to capture semantic information even for rare or misspelled words.

FastText can generate embeddings for out-of-vocabulary words by averaging the embeddings of its constituent character n-grams. This approach improves the robustness of word representations, particularly for morphologically rich languages and tasks involving rare or misspelled words. Additionally, FastText is efficient and scalable, making it suitable for large-scale text-processing tasks.
Although FastText has its own Python library, in this practice, we will use FastText models throughout the Gensim library.

GloVe, similar to Word2Vec, is an unsupervised learning algorithm used to generate word embeddings. It learns vector representations for words by examining the global co-occurrence statistics of words within a corpus. The key idea behind GloVe is to

capture the relationship between words by considering their co-occurrence probabilities in the entire corpus. It constructs a global word-word co-occurrence matrix where each element represents the number of times two words co-occur in a context window.

GloVe then factorizes this co-occurrence matrix to obtain word embeddings. By minimizing a specially designed objective function that incorporates both global co-occurrence statistics and vector space geometry, GloVe learns word embeddings that effectively capture semantic relationships between words. The generated word-embedding vectors are known for their ability to encode meaningful semantic information and exhibit robust performance across a wide range of NLP tasks.

Gensim is a library for topic modeling, document indexing and retrieval of large data collections. The code for this is available at https://pypi.org/project/gensim/ and https://radimrehurek.com/gensim/index.html. This library also provides functionalities to work with static word embeddings.

Installing Gensim library
This library is available on the python.

Using pip
```
pip install gensim
pip install --upgrade gensim # to upgrade version
```

Using conda
```
conda install -c conda-forge gensim
```

Using source package
If you have downloaded the source package, you need to unzip it and run the following commands.
```
python setup.py test
python setup.py install
```

EXAMPLE 1: Listing the pre-trained word-embeddings in Gensim

```
import gensim.downloader as api
info = api.info()
for item in info["models"].items():
  print(item)
```

Example 2 shows how to load three distinct pre-trained word embedding representations constructed using the Word2Vec, GloVe, and FastText methods. Notice that these pretrained embeddings are all available in the Gensim library.

EXAMPLE 2: Loading pre-trained word embedding vectors
Loading the pre-trained word embedding vectors generated by the Word2Vec method trained on Google News (about 100 billion words).

```
model = api.load("word2vec-google-news-300")
vec = model.get_vector("husband")
print(vec)
```

Loading the pre-trained word embedding vectors generated by the FastText method trained on Wikipedia 2017, UMBC webbase corpus and statmt.org news dataset.

```
model = api.load("fasttext-wiki-news-subwords-300")
vec = model.get_vector("husband")
print(vec)
```

Loading the pre-trained word embedding vectors created by the GloVe method trained on Wikipedia 2014+ Gigaword 5.

```
model = api.load("glove-wiki-gigaword-300")
vec = model.get_vector("husband")
```

```
print(vec)
```

The third example illustrates how to use the word embedding vectors *word2vec-google-news-300*. The semantic relationship between the words *man, woman, kitchen,* and *football* can be captured. The results of the cosine similarity among the word vectors are shown in Table 1.

	man	*woman*	*kitchen*	*football*
man	1.000	0.766	0.077	0.268
woman	0.766	1.000	0.138	0.121
kitchen	0.077	0.138	1.000	0.054
football	0.268	0.121	0.054	1.000

Table 1: Cosine similarity of the words *man, woman, kitchen,* and *football* based on the pre-trained word vectors *word2vec-google-news-300.*

Table 1 shows that the words *woman* and *man* have the highest similarity, with a value of 0.766, which is expected. However, some bias can be noticed in the relationship between *woman* and *kitchen*, which shows a similarity of 0.138 compared to *man* and *kitchen*, which has a lower value of 0.077. A similar issue is evident between *man* and *football*, with a similarity of 0.288 compared to *woman* and *football*, which is 0.121. The most dissimilar pair observed is *kitchen* and *football*.

EXAMPLE 3: Computing similarity between words based on the pre-trained vector representations

```
import numpy as np
from sklearn.metrics.pairwise import cosine_similarity
words = "man woman kitchen football".split()
#Obtain the vectors for each word
dv=[model.get_vector(word) for word in words]
sim= np.round(cosine_similarity(dv,dv), 3)
dn = pd.DataFrame(index=words, columns=words, data=sim)
```

Static word embedding models can capture some **semantic relationships** between words. Using the `most_similar()` method users can ask the model about relations such as:

- Sweden is similar to …

```
model.most_similar(positive=["Sweden"], negative=[],
topn=15, restrict_vocab=None, indexer=None)
```

- Man is to king as woman is to …

```
model.most_similar(positive=["king", "woman"],
negative=["man"], topn=15, restrict_vocab=None,
indexer=None)
```

- Madrid is to Spain as Rome is to …

```
model.most_similar(positive=["Spain", "Rome"],
negative=["Madrid"], topn=15, restrict_vocab=None,
indexer=None)
```

Loading local pre-trained word embeddings based on Word2Vec or FastText

The models available in the GENSIM API are mostly for English. However, this library provides methods to load a pre-trained embedding matrix from a local file. Loading other word embedding vectors can be useful, trained on distinct corpora or in other languages. For that, the **KeyedVector** class can be used. The pretrained embedding vectors generated by both methods can be exported in text or binary format. Then, it is important to know the format in which the pretrained embedding vectors were saved.
The first step is downloading the pre-trained word embedding vectors we want to use to a local file. The second step is to load the local file using the class **KeyedVector.**

EXAMPLE 4: Loading pre-trained word embedding (created by Word2Vec) from a local file

```
#Download pre-trained word embedding vectors generated by Word2Vec trained on Google News 2013 dataset
!wget http://vectors.nlpl.eu/repository/20/1.zip
!unzip 1.zip
from gensim.models.keyedvectors import KeyedVectors

#Loading the pre-trained word embedding vectors from a binary file format
wordvb = KeyedVectors.load_word2vec_format('model.bin', binary=True, limit=None)

#Loading the pre-trained word embedding vectors from a text file format
wordvt = KeyedVectors.load_word2vec_format('model.txt', binary=False, limit=None)
```

EXAMPLE 5: Loading pre-trained word embedding (created by FastText) from a local file

```
#Download pre-trained word embedding vectors generated by FastText trained on Common Crawl and Wikipedia, in binary format
!wget  https://dl.fbaipublicfiles.com/fasttext/vectors-crawl/cc.en.300.bin.gz
from gensim.models.keyedvectors import KeyedVectors
#Loading the pre-trained word embedding vectors from a binary file format
wordvb                                                    =
KeyedVectors.load_word2vec_format('cc.en.300.bin.gz',
binary=True, limit=None)

#Download pre-trained word embedding vectors generated by FastText trained on Common Crawl and Wikipedia, in text format
!wget  https://dl.fbaipublicfiles.com/fasttext/vectors-crawl/cc.en.300.vec.gz
#Loading the pre-trained word embedding vectors from a text file format
wordvt                                                    =
KeyedVectors.load_word2vec_format('cc.en.300.vec.gz',
binary=False, limit=None)
```

Loading local pre-trained word embeddings based on GloVe

The class KeyedVector also has the capability to load word embeddings generated by GloVe. In this scenario, it is essential to set the parameter *no_header=True*.

EXAMPLE 6: Loading pre-trained word embedding (created by Glove) from a local file

```
#Download the pre-trained word embedding vectors generated by GloVe trained on a Twitter corpus
!wget https://nlp.stanford.edu/data/wordvecs/glove.twitter.27B.zip
!unzip glove.twitter.27B.zip
from gensim.models.keyedvectors import KeyedVectors
size=None
wordv = KeyedVectors.load_word2vec_format('glove.twitter.27B.200d.txt', binary=False, no_header=True, limit=size)
```

Pre-trained word embedding vectors in other languages

Pre-trained word embedding vectors are available in English and various other languages. For instance, researchers and developers have created pre-trained word embedding vectors for languages such as Spanish, French, German, Chinese, and many others. The two links below offer access to pre-trained word embedding vectors, which vary in language, training corpus, algorithms, parameters, and other characteristics.

- http://vectors.nlpl.eu/repository/
- https://fasttext.cc/docs/en/crawl-vectors.html

In practical terms, accessing and utilizing these word embeddings in different languages is made convenient through libraries like Gensim. Gensim provides the **KeyedVector** class, which offers seamless integration for loading and working with pre-trained word embedding vectors. Developers can leverage this class to load and manipulate embeddings effortlessly regardless of the English, Spanish, or any other language.

EXAMPLE 7: Loading pre-trained word embedding vectors in Spanish from a local file

In the case of the Spanish language, in the link https://dl.fbaipublicfiles.com/fasttext/vectors-crawl/cc.es.300.bin.gz a pre-trained word embedding vectors can be found. It was created using the FastText method.

```
#Download pre-trained word embedding vectors generated
by FastText trained on Common Crawl and Wikipedia, in
binary format
!wget https://dl.fbaipublicfiles.com/fasttext/vectors-
crawl/cc.es.300.bin.gz
from gensim.models.keyedvectors import KeyedVectors
#Loading the pre-trained word embedding vectors from a
binary file format
wordvb =
KeyedVectors.load_word2vec_format('cc.es.300.bin.gz',
binary=True, limit=None)
```

Training word embedding vectors using Gensim

Gensim provides functionalities to train custom word embedding vectors tailored to the characteristics of a specific corpus. With Gensim, users can train their word embeddings using techniques like Word2Vec or FastText. This enables users to capture their domain-specific data's semantic relationships and contextual nuances, leading to more accurate and context-aware language understanding systems.

EXAMPLE 8: Learning word embedding vectors on a specific corpus

```
import nltk
nltk.download("punkt")
from gensim.models import Word2Vec
from nltk.tokenize import word_tokenize
# Tokenize the corpus. Notice that the preprocessing
and tokenization of the texts can be customized
depending on the task.
tokenized_corpus = [word_tokenize(sentence.lower()) for
sentence in df["text"].tolist()]
# Train the Word2Vec model
model = Word2Vec(sentences=tokenized_corpus,
vector_size=100, window=5, min_count=1, workers=4)
# Get the word vector for a specific word
word_vector = model.wv["woman"]
# Most similar words to "example"
similar_words = model.wv.most_similar("woman")
print("Word vector for 'woman':", word_vector)
print("Most similar words to 'woman':", similar_words)
# Save the trained word embeddings vectors
model.wv.save_word2vec_format("my-word-embedding-
vectors-100d.txt", binary=False)
```

Obtaining the representation of texts utilizing the embedded word vectors it contains

Once you obtain the words in a text and have a word embedding matrix, it is possible to represent a sentence or document using the vectors of the words in it. A straightforward way to get the sentence representation is by averaging the vector of all words in it.

EXAMPLE 9: Obtaining the representation of a text using word embedding vectors.

```
#Obtaining the words or terms in the text. This time,
we considered simple whitespace splitting.
#Notice that, after split the texts, the stopwords can
be removed
#dim is the dimension of the word embedding vectors.
def get_sentence_rep(text, keyedvec, dim=300):
   words= text.lower().split()
   zero_vec=np.zeros(dim)
   avg_vec=np.zeros(dim)
   total_w=0
   for w in words:
       try:
           avg_vec+=keyedvec[w]
           total_w+=1
       except:
           print(f"The word {w} is not found in the
embedding matrix")
           pass

   if total_w == 0:
       return zero_vec
   return avg_vec/total_w

text= "The theory of paint indicates feelings of
isolation in society"
get_sentence_rep(text, wordv, dim=200)
```

ACTIVITY

Implement text representations for the corpora *EXIST2024_EN_examples.csv (English language) and EXIST2024_ES_examples.csv (Spanish language)*. Each text in the corpus should be represented using three distinct **static embedding** for English and one **static embedding** for Spanish.

ENGLISH

- *word2vec-google-news-300*
- *fasttext-wiki-news-subwords-300*
- *glove-wiki-gigaword-300*

SPANISH

- *Fasttext (https://dl.fbaipublicfiles.com/fasttext/vectors-crawl/cc.en.300.vec.gz)*

The representation of each text using static embeddings should be the average vector of the words it contains, applying the following preprocessing:

- remove URLs.
- remove hashtags.
- remove user references.
- lowercase.
- tokenize (using *word_tokenize*).
- remove stopwords (using *nltk stopwords*).

Note that tokenization and stopword removal are language-dependent.

It is important to emphasize that the most similar tweet pairs must be determined **separately** for each class: **'YES'** and **'NO'**. Specifically, you should find the most similar pair of tweets among those classified as 'SEXIST' and another most similar pair among those classified as 'NON-SEXIST'. To measure the similarity between tweet representations, you must use the *cosine_similarity* function provided by scikit-learn.